Project Monitoring and Control

Chapter_6

What is EVM

- EVM is a set of measurements that compares:
  - **What you planned to do by today**
  - **What you actually completed by today**
  - **What you actually spent by today**
- It uses a baseline plan (scope + schedule + budget) and converts progress into money value.

Why EVM is used

- EVM is used because it gives early, objective signals about project health.
  - It detects schedule and cost problems early (often before people “feel” the project is failing).
  - It avoids false comfort, like “We used only 40% budget” when you completed only 20% of work.
  - It gives a disciplined way to forecast final cost and finish date.

When to use EVM

- Use EVM when the project has:
  - A defined scope and deliverables (WBS exists).
  - A planned schedule with milestones.
  - A budget broken down by work packages.
  - Enough duration and complexity that tracking matters.
- Typical use cases:
  - Software development projects with multiple modules and phases.
  - Construction and infrastructure projects.
  - Government and donor-funded projects with strict reporting.
  - Large campus systems such as EMIS, ERP, network deployment.
- Avoid or keep it lightweight when:
  - The project is very small or very short.
  - Scope changes daily without control.
  - You cannot measure progress reliably.

Where EVM fits in the project lifecycle

- EVM is mainly used in **Execution and Monitoring/Controlling**, but it depends on good planning.
  - **Initiation**: decide if EVM will be used.
  - **Planning**: build WBS, schedule, cost baseline (this is essential).
  - **Execution**: collect actual cost, measure progress.
  - **Monitoring/Control**: calculate EVM metrics, take corrective actions.
  - **Closing**: compare planned vs actual, record lessons learned.

How EVM works, step by step

- **Step 0: Create the baseline (this is mandatory)**
- You cannot do EVM without a baseline.
- You need:
  1. **WBS** (Work Breakdown Structure): break work into work packages.
  2. **Schedule**: planned start and finish for each work package.
  3. **Budget**: assign cost to each work package.
  4. **Measurement rule**: define how you will measure progress (0/100, 50/50, percent complete, milestone-based, etc.)
- The result is your **Performance Measurement Baseline (PMB)**.

Step 1: Define the three core values (PV, EV, AC)

- At every reporting date (weekly or biweekly), collect:
- **A) Planned Value (PV)**
- PV is the budgeted value of the work that should have been done by today.
- Think: “According to the plan, how much work should be completed by now, in money terms?”
- PV is also called **BCWS**.
- **B) Earned Value (EV)**
- EV is the budgeted value of the work that is actually completed by today.
- Think: “Based on actual progress, how much planned budget have we earned?”
- EV is also called **BCWP**.
- **C) Actual Cost (AC)**
- AC is what you have actually spent by today for the work done.
- AC is also called **ACWP**.

Step 2: Compute schedule and cost variances (SV, CV)

- These tell you if you are behind or ahead.
- **Schedule Variance (SV)**
  - SV = EV − PV
  - SV > 0: ahead of schedule
  - SV = 0: on schedule
  - SV < 0: behind schedule
- **Cost Variance (CV)**
  - CV = EV − AC
  - CV > 0: under budget
  - CV = 0: on budget
  - CV < 0: over budget

Step 3: Compute performance indices (SPI, CPI)

- Variances are in money units. Indices show performance ratios, which are easier to compare.
- **Schedule Performance Index (SPI)**
  - SPI = EV / PV
  - SPI > 1: ahead of schedule
  - SPI = 1: on schedule
  - SPI < 1: behind schedule
- **Cost Performance Index (CPI)**
  - CPI = EV / AC
  - CPI > 1: under budget (good)
  - CPI = 1: on budget
  - CPI < 1: over budget (bad)

Step 4: Forecast the final cost (EAC) and remaining work (ETC)

- Forecasting is one of the biggest reasons EVM is used.
- Let:
- **BAC** = Budget at Completion (total planned budget)
- Common formulas:
- **Estimate at Completion (EAC)**
- If current cost efficiency continues:
  - EAC = BAC / CPI
- If you expect remaining work to follow original plan (rare):
  - EAC = AC + (BAC − EV)
- **Estimate to Complete (ETC)**
- ETC = EAC − AC

Step 5: Interpret results and take control actions

- EVM numbers are not for decoration. They must lead to action, such as:
  - Re-allocating resources to delayed work packages
  - Removing blockers
  - Reducing scope through formal change request
  - Negotiating timeline changes
  - Improving productivity, quality, or procurement plans

6) A complete worked example (clear and realistic)

- **Scenario**
  - A web-based “Student Attendance and Class Log System” project has:
  - Total budget (BAC): **NPR 200,000**
  - Planned duration: 10 weeks
  - Today is end of week 5 (halfway)
- According to plan, you should complete **50%** of work by now.
- So:
  - PV = 50% of 200,000 = **NPR 100,000**
- Actual progress check shows only **40%** of scope is completed.
- So:
  - EV = 40% of 200,000 = **NPR 80,000**

- Accounting shows you spent **NPR 120,000** so far.
- So:
  - AC = **NPR 120,000**
- Now calculate:
- **Variances**
  - SV = EV − PV = 80,000 − 100,000 = **−20,000** (behind schedule)
  - CV = EV − AC = 80,000 − 120,000 = **−40,000** (over budget)
- **Indices**
  - SPI = EV / PV = 80,000 / 100,000 = **0.80** (slow progress)
  - CPI = EV / AC = 80,000 / 120,000 = **0.67** (poor cost efficiency)

- **Forecast final cost**
  - EAC = BAC / CPI = 200,000 / 0.67 ≈ **298,507** (approx.)
  - Meaning: if this continues, the project may finish around **NPR 300,000**, not 200,000.
  - ETC = EAC − AC ≈ 298,507 − 120,000 ≈ **178,507**
- **Interpretation in plain words**
  - At week 5:
  - You are behind schedule (40% done instead of 50%).
  - You are over budget (spent 120k, but earned only 80k value).
  - If nothing changes, you may need about 100k extra to finish.

7) How to measure EV correctly (the practical "how")

- The biggest challenge is measuring "percent complete" honestly.
- Better approaches:
- **A) Milestone-based earning (best for software)**
- Example for a module worth NPR 50,000:
  - UI complete: 20%
  - API complete: 30%
  - Testing complete: 30%
  - Deployment + documentation: 20%
- You only "earn" the portion that is truly finished.

- **B) 0/100 rule**
  - You earn 0% until the work package is complete, then earn 100%.
    Good for short tasks where partial credit is misleading.
- **C) 50/50 rule**
  - You earn 50% when work starts, and 50% when it finishes.
    Useful when tasks are medium-sized.
- **D) Percent complete (use carefully)**
  - Only if you have objective evidence (test cases passed, review accepted, etc.)

Advantages of EVM

- Combines scope, time, and cost in one system
- Early warning for schedule and budget problems
- Makes progress reporting more objective
- Supports forecasting (EAC, ETC)
- Helps management take corrective action with evidence
- Strong for reporting to stakeholders (donors, management, steering committee)

Disadvantages and limitations

- Needs strong planning (WBS, schedule, cost baseline)
- Requires accurate progress measurement, which is hard in software if poorly defined
- If scope changes frequently without change control, EVM becomes unreliable
- Can create “reporting pressure” and biased progress reporting if culture is weak
- More overhead for small projects

Monitoring Techniques and Tools

- Project monitoring means collecting reliable facts about progress, cost, quality, and risks, then comparing them with the plan so you can act early.
- It is not supervision or fault-finding.
- It is a control loop: plan, measure, compare, correct.

What it is

- **Project monitoring** is the continuous process of tracking:
  - Work progress against schedule (tasks, milestones, deliverables)
  - Cost against budget (planned vs actual spending)
  - Quality against standards (defects, rework, acceptance results)
  - Scope stability (change requests, requirement churn)
  - Risks and issues (what might happen, what is happening)
  - Resources (availability, utilization, productivity)
- The output of monitoring is decision-ready information, not raw data.

Why monitoring is needed

- Monitoring helps you:
  - Detect slippage early, before deadlines are missed
  - Control cost growth before it becomes unmanageable
  - Reduce rework by catching quality problems early
  - Keep stakeholders aligned with facts, not assumptions
  - Identify bottlenecks and unblock the team
  - Support fair reporting and accountability
- Without monitoring, a project often “looks fine” until it suddenly fails.

When to monitor

- Monitoring is not a one-time activity. It should be done:
  - Daily for critical operational signals (task flow, blockers)
  - Weekly for progress, cost, and issues (most common)
  - At every milestone or phase gate (formal review)
  - Whenever a major change request arrives (impact analysis)
  - After major incidents (defect spikes, vendor delay, staff change)
- For a 4–12 week student or department project, weekly monitoring is usually enough. For large projects, monitoring may be daily plus weekly governance.

Where monitoring is applied

- Monitoring is used across domains:
  - Software projects: sprints, releases, defect trends, deployment readiness
  - Construction: materials, labor productivity, inspection results
  - Network deployments: site readiness, device installation, connectivity tests
  - University systems (EMIS/ERP): module-wise progress, training status, data migration completion
- Monitoring also happens at different levels:
  - Team level (developer tasks, test cases)
  - Project manager level (schedule, budget, risks)
  - Sponsor level (milestones, benefits, major issues)

How monitoring works, step by step

- **Step 1: Define what you will measure (metrics)**
- You cannot monitor everything. Choose a small, useful set:
- **Schedule**
- Planned vs completed tasks
- Milestone completion rate
- SPI (if using EVM) or schedule variance
- **Cost**
- Actual spending vs budget
- CPI (if using EVM)
- Burn rate per week

Scope
•Number of change requests
•Requirement churn percentage
Quality
•Defects found per week
•Defect leakage (bugs found after release)
•Rework percentage
Risk and Issues
•Count of open issues, aging of issues
•High risks and mitigation status

- **Step 2: Collect data consistently**
- Use clear rules:
  - Progress is accepted only when it meets “done” criteria (tested, reviewed, documented, accepted)
  - Costs are collected from actual invoices, payroll estimates, or purchase records
  - Issues are logged with owner, deadline, and priority

- **Step 3: Compare with the baseline plan**
  - This is where monitoring becomes meaningful:
  - Are we ahead or behind schedule?
  - Are we over or under budget?
  - Is quality improving or getting worse?
  - Are risks increasing or decreasing?

- **Step 4: Analyze causes, not just symptoms**
- Examples:
  - “Coding is slow” is a symptom.
  - Root causes might be unclear requirements, unstable internet, new team members, or poor module integration.
- Use simple techniques:
  - 5 Whys
  - Fishbone (cause-effect)
  - Pareto (top causes)

- **Step 5: Take corrective or preventive actions**
- Monitoring without action is useless.
- Corrective actions:
  - Add resources to critical tasks
  - Remove blockers, clarify requirements
  - Re-plan a milestone
  - Improve testing and review discipline
- Preventive actions:
  - Extra training
  - Better checklists
  - Earlier code reviews
  - Vendor backup plan

- **Step 6: Communicate with the right format**
- Different audiences need different detail:
- Team: task-level, daily blockers
- Sponsor: milestone status, major risks, cost trend
- Client/user: deliverables, acceptance status, change requests

6) Monitoring techniques (methods) with examples

- **1. Status meetings (daily or weekly)**
- Purpose: quick alignment and escalation.
- Good practice:
  - Fixed agenda: progress, next steps, blockers, decisions needed
  - Time-boxed: 15 minutes daily or 30–60 minutes weekly
  - End with actions and owners
- Example (software project):
  - Monday review shows “Login module done” but testing blocked due to missing test accounts. Action: admin creates accounts by today.

- **2. Work performance reports**
- Purpose: formal, written record.
- Typical sections:
  - Completed work this period
  - Planned work next period
  - Variance (schedule/cost)
  - Risks/issues and decisions needed
- Example (university system implementation):
  - “Data migration is 70% complete, but department A is not providing verified data. Risk: inaccurate records. Decision needed: assign focal person.”

- **3. Milestone reviews and phase gates**
- Purpose: confirm readiness before moving to next stage.
- Common gates:
  - Requirements signed off
  - Design approved
  - UAT completed
  - Deployment authorized
- Example:
  - For EMIS module rollout, you do not start training until the module passes UAT with at least 95% critical test cases passed.

- **4. Inspection and quality audits**
- Purpose: ensure deliverables meet standards.
- Examples:
  - Code review audit: check if peer review happened for all merges
  - Document audit: verify version control, approvals, and completeness

- **5. Trend analysis**
- Purpose: understand direction over time, not just today’s status.
- Examples:
  - Defect trend: bugs per week should decrease near release
  - Velocity trend: sprint completion stability
  - Cost burn trend: spending rate vs plan

- **6. Earned Value tracking (if used)**
- Purpose: objective measurement of cost and schedule performance.
- Use when there is a baseline and measurable scope.

7) Tools used in monitoring

- **A) Scheduling and task tracking tools**
- Microsoft Project: baseline schedule, dependencies, critical path
- Jira: sprint boards, backlog, burndown, issue tracking
- Trello/Asana: lightweight task tracking for small projects
- Best for:
- Task progress, due dates, ownership, blockers

- **B) Dashboards and reporting tools**
- Power BI, Google Looker Studio, Excel dashboards
- Simple KPI dashboard in Sheets
- Best for:
- Sponsors and management who need a fast overview

- **C) Documentation and collaboration**
- Confluence, Google Drive, SharePoint
- Version control for docs (when needed)
- Best for:
- Meeting minutes, decisions, change logs, lessons learned

- **D) Version control and CI tools (software projects)**
- GitHub/GitLab, Bitbucket
- CI pipelines: GitHub Actions, GitLab CI, Jenkins
- Best for:
- Tracking work completion via pull requests, build status, test results

- **E) Communication tools**
  - Email, Teams, Slack, WhatsApp (for quick updates)
  - Use carefully: decisions must be recorded formally
- **F) Risk and issue tracking tools**
  - Issue log in Excel/Sheets
  - Jira issue types for risk and issue
  - Simple Kanban board: To Do, In Progress, Resolved
- Best for:
  - Clear ownership and follow-up, no missed problems

Advantages of strong monitoring

- Better control of time and cost
- Fewer surprises near deadlines
- Higher quality and less rework
- Better stakeholder trust and transparency
- Faster decision-making and escalation
- Better learning for future projects (lessons learned)

Disadvantages and common mistakes

- **Disadvantages**
  - Adds management overhead (time and documentation)
  - Can become “reporting for reporting’s sake” if poorly designed
  - Too many metrics can confuse everyone
- **Common mistakes**
  - Measuring activity instead of completion (hours worked instead of deliverables accepted)
  - Inflated progress reporting (claiming 90% done for weeks)
  - No clear “definition of done”
  - Issues tracked but not owned and not closed
  - Meetings without action items
  - Tools used, but data not updated regularly

Case: Campus network upgrade project (6 weeks)

- Plan:
  - Week 1–2: procurement and cabling
  - Week 3–4: switch configuration and installation
  - Week 5: testing and documentation
  - Week 6: training and handover
- Monitoring setup:
  - Weekly milestone review with checklist
  - Issue log (owner, priority, due date)
  - Gantt chart updated weekly
  - Testing checklist (connectivity, VLAN, throughput)

- Week 3 monitoring findings:
  - Cabling delayed due to vendor shortage.
  - Two buildings not ready (civil work incomplete).
    Actions:
  - Re-sequence: configure switches in lab first
  - Escalate: campus admin completes readiness by Friday
  - Adjust schedule baseline with approved change request
- Result:
  - Delay reduced from two weeks to five days because monitoring revealed the real blocker early.

Change Control Management,

- Change Control Management is the disciplined way of handling changes in a project so that useful changes are accepted and harmful or uncontrolled changes are avoided.
- In simple words, it answers this question: **What happens when someone asks to change the plan?**

What is Change Control Management

- Change Control Management is a formal process used to **identify, evaluate, approve or reject, and implement changes** to a project's scope, schedule, cost, quality, or resources.
- A change can be:
  - Adding new features
  - Modifying requirements
  - Changing deadlines
  - Adjusting budget
  - Replacing tools or technologies
  - Changing team members or vendors
- Change control ensures that every change is **understood before it is done**.

Why Change Control Management is necessary

- Projects rarely run exactly as planned. New needs appear, mistakes are discovered, and external conditions change. Without change control:
  - Scope grows silently (scope creep)
  - Costs increase without approval
  - Deadlines slip without explanation
  - Quality drops due to rushed work
  - Team morale suffers due to confusion
- Change control protects the project by making changes **visible, evaluated, and approved**.

When Change Control is used

- Change control is used **throughout the project**, mainly during execution and monitoring.
- Typical situations:
  - Client requests a new feature after development has started
  - Management wants faster delivery without extra resources
  - A regulation or policy changes mid-project
  - A technical solution proves unworkable
  - Budget cuts or extensions occur
- Important point:
  Not every suggestion is a change request. Only changes that affect **baseline elements** (scope, time, cost, quality) require formal change control.

Where Change Control applies

- Change control applies to:
  - **Scope baseline**: requirements, features, deliverables
  - **Schedule baseline**: task dates, milestones
  - **Cost baseline**: budget allocations
  - **Quality standards**: acceptance criteria
  - **Resources**: people, tools, vendors
  - **Documentation**: plans, designs, contracts
- In IT projects, change control is especially important because small changes can have large downstream effects.

Types of changes

- **Scope change**
  Adding, removing, or modifying features or requirements.
- **Schedule change**
  Changing deadlines, milestones, or task sequence.
- **Cost change**
  Increasing or decreasing budget.
- **Technical change**
  Switching technology, platform, or architecture.
- **Resource change**
  Changing team members, skills, or vendors.
- **Quality change**
  Modifying acceptance criteria or performance standards.

How Change Control works, step by step

- **Step 1: Change request submission**
- A change begins with a **formal change request**, not verbal instruction.
- A change request should include:
  - Description of the change
  - Reason for the change
  - Who requested it
  - Date and urgency
  - Affected areas (scope, time, cost, quality)
- Example:
  “Add SMS notification feature to student attendance system so parents receive daily updates.”

- **Step 2: Change logging**
- The change request is recorded in a **change log** so it is tracked and not forgotten.
- Typical change log fields:
- Change ID
- Description
- Requester
- Status (submitted, under review, approved, rejected)
- Decision date

Step 3: Impact analysis

- This is the most critical step.
- The project team analyzes:
  - **Schedule impact**: extra days or weeks required
  - **Cost impact**: additional budget needed
  - **Resource impact**: extra staff or skills required
  - **Quality impact**: risk of defects or rework
  - **Risk impact**: new risks introduced
- Example analysis:
  - SMS feature requires 2 extra weeks
  - Additional cost: NPR 25,000
  - Requires API integration and testing
  - Adds risk of message delivery failure

Step 4: Review and decision

- The change is reviewed by the appropriate authority:
  - Project Manager (small changes)
  - Change Control Board (medium to large projects)
  - Sponsor or client (contractual changes)
- Possible decisions:
  - Approved
  - Approved with conditions
  - Deferred
  - Rejected
  - The decision must be documented.

- **Step 5: Baseline update**
- If approved:
- Update scope documents
- Revise schedule and budget
- Update risk register
- Communicate changes to stakeholders
- This step is essential. A change without baseline update leads to confusion later.

- **Step 6: Change implementation**
  - The approved change is executed according to the updated plan.
  - Assign tasks
  - Track progress
  - Test and validate
  - Ensure quality standards are met
- **Step 7: Verification and closure**
  - After implementation:
  - Verify that the change achieved its objective
  - Obtain acceptance if required
  - Close the change request in the change log

Example Scenario: College EMIS implementation

- Original plan:
  - Modules: student record, attendance, exam result
  - Budget: NPR 500,000
  - Duration: 4 months
- Midway request:
  - Add online fee payment module
- Impact analysis shows:
  - Extra 3 weeks required
  - Additional cost: NPR 80,000
  - Requires payment gateway integration
  - Introduces security and compliance risks

- Decision:
  - Approved by management with extra budget and revised deadline
- Actions:
  - Schedule updated
  - Budget revised
  - Security testing added
  - Change documented and tracked
- Result:
  - System delivered with clear agreement, no disputes at handover

Advantages

- Prevents uncontrolled scope creep
- Maintains budget and schedule discipline
- Improves transparency and accountability
- Supports informed decision-making
- Reduces conflict with clients and stakeholders
- Improves project predictability and documentation

Disadvantages and limitations

- Adds paperwork and approval time
- Can slow down decision-making if overly bureaucratic
- May frustrate stakeholders who want quick changes
- Requires disciplined documentation and follow-up

- Poorly implemented change control can become an obstacle instead of support.

Common mistakes in Change Control

- Accepting changes verbally without documentation
- Skipping impact analysis
- Approving changes without adjusting budget or timeline
- Implementing changes before approval
- Not communicating approved changes to the team
- Treating every small suggestion as a major change

Change Control vs Scope Creep

- **Change Control**: planned, evaluated, approved, documented change
- **Scope Creep**: uncontrolled, informal, undocumented change
- Good projects allow change, but **only through control**.

Issue and Risk Tracking

- Issue and risk tracking is the practical system a project uses to make sure problems are not ignored, repeated, or forgotten.
- It is one of the most important parts of project control because most projects fail slowly through small, unmanaged issues and risks.

What is Issue Tracking

- An **issue** is a problem that has already happened and is affecting the project now.
- Examples of issues:
  - Developer cannot access the server (work stopped).
  - Vendor did not deliver switches on time (installation delayed).
  - Key staff member is absent (task ownership missing).
  - A module fails testing (rework needed).
- **Issue tracking** is the process of:
  - Logging the issue
  - Assigning an owner
  - Setting priority and due date
  - Monitoring progress
  - Closing it with a resolution

What is Risk Tracking

- A **risk** is a possible future event that can impact the project if it happens.
- Examples of risks:
  - Risk of internet outage during online examination week.
  - Risk that user training will be insufficient.
  - Risk that requirements will change after development starts.
  - Risk that data migration will be inaccurate.
- **Risk tracking** is the process of:
  - Identifying and recording risks
  - Estimating probability and impact
  - Planning responses (mitigation, avoidance, transfer, acceptance)
  - Monitoring risk triggers and response actions
  - Updating risk status over time

Why issue and risk tracking are necessary

- Without tracking:
  - Problems repeat because lessons are not recorded.
  - Small delays grow into major failure.
  - Stakeholders lose trust because surprises keep happening.
  - Team wastes time in confusion about who is solving what.
  - Decisions are taken late, when options are limited.
- With tracking:
  - Everyone knows what is open, what is urgent, and who owns it.
  - You can act early, not after damage is done.
  - Project status becomes fact-based.

When to track issues and risks

- **Issue tracking frequency**
  - Updated whenever a new issue is found
  - Reviewed daily (fast projects) or weekly (most projects)
- **Risk tracking frequency**
  - Updated weekly or at key milestones
  - Updated immediately when:
    - New risks appear
    - A risk becomes an issue
    - A major change request arrives

Where tracking is used

- Issue and risk tracking is used in:
  - Software projects (bugs, integration problems, testing failures)
  - Network projects (site readiness, device failure, cable issues)
  - Construction projects (materials shortage, weather delays)
  - University systems (EMIS/ERP adoption, training gaps, data accuracy)
- It is also used at different levels:
  - Team level: technical issues, task blockers
  - Project level: schedule, vendor, quality, scope risks
  - Sponsor level: budget and delivery risks, policy constraints

How to track issues, step by step

- **Step 1: Create an Issue Log**
- An issue log can be in Excel/Google Sheets, Jira, Trello, or MS Project.
- Minimum fields:
  - Issue ID
  - Date reported
  - Description
  - Category (schedule, cost, quality, vendor, technical, user)
  - Priority (High/Medium/Low)
  - Impact (what it affects)
  - Owner (responsible person)
  - Due date
  - Status (Open/In progress/Resolved/Closed)
  - Resolution notes

- **Step 2: Record the issue clearly**
  - A good issue description is specific.
  - Poor: “System is not working.”
    Better: “Attendance module fails to save records when internet disconnects, error code 504.”
- **Step 3: Prioritize and assign ownership**
  - Use simple priority rules:
  - High: stops work or impacts deadline
  - Medium: workaround exists, but harms quality or cost
  - Low: minor inconvenience, no major impact
  - Every issue must have one owner, not “team.”

- **Step 4: Decide the immediate action**
  - For each issue:
  - Fix now
  - Apply workaround
  - Escalate to sponsor or vendor
  - Defer with justification
- **Step 5: Review and close properly**
  - Do not close an issue until:
  - The solution is verified
  - The impact is removed
  - The resolution is documented

How to track risks, step by step

- **Step 1: Create a Risk Register**
- Minimum fields:
  - Risk ID
  - Risk statement (cause → event → impact)
  - Probability (Low/Medium/High or 1–5)
  - Impact (Low/Medium/High or 1–5)
  - Risk score (Probability × Impact)
  - Risk owner
  - Response strategy (avoid/mitigate/transfer/accept)
  - Mitigation actions
  - Triggers (warning signs)
  - Status (Open/Monitoring/Closed)
  - Notes

- **Step 2: Write the risk properly (cause → event → impact)**
- Example:
  "Due to limited training time (cause), staff may use EMIS incorrectly (event), leading to inaccurate student data and rework (impact)."
- This format prevents vague risks.
- **Step 3: Assess probability and impact**
- Simple scale:
  - Probability: 1 (rare) to 5 (very likely)
  - Impact: 1 (small) to 5 (severe)
  - Risk score = P × I
    Focus on the highest scores first.

Step 4: Choose a response strategy

- **Avoid**: change plan to remove the risk
  Example: avoid unstable hosting by using a reliable cloud provider.
- **Mitigate**: reduce probability or impact
  Example: early training, pilot testing, backup internet.
- **Transfer**: shift risk to a third party (contract, insurance)
  Example: vendor warranty, SLA with service provider.
- **Accept**: do nothing beyond monitoring
  Example: low impact risks.

- **Step 5: Define triggers and monitor**
- A trigger is an early warning sign.
- Example triggers:
- Training attendance below 70%
- Defect rate rising for two consecutive weeks
- Vendor delivery slipping by more than 3 days
- Requirements changed more than twice in one month

- **Step 6: Convert risks into issues when they happen**
- If a risk event occurs, it becomes an issue and must move into the issue log.
- Example:
  Risk: “Internet outage may interrupt online demo.”
  If outage happens on demo day, it becomes an issue: “Demo interrupted due to outage.”

- **For professional projects**
- Jira (best for software)
- Azure DevOps Boards
- GitHub Issues
- MS Project (basic issue log)
- Service desk tools (for IT operations)

Advantages

- Clear accountability (owner and due date)
- Faster problem resolution
- Fewer repeated mistakes
- Better stakeholder confidence
- Helps in audit and reporting
- Improves planning accuracy over time

Disadvantages and common mistakes

- **Disadvantages**
  - Requires discipline and time to update
  - Can become paperwork if not used for decisions
- **Common mistakes**
  - Logging issues without assigning an owner
  - No due dates, so issues never finish
  - Closing issues without verification
  - Treating risks as “fear list” without mitigation actions
  - Keeping risks hidden to “look good” in reports

Web-based Attendance and Teaching Log System for multiple college

- **Top risks**

1. Risk: Teachers may resist using the system
   Mitigation: short training + very simple UI + mobile-friendly page
   Trigger: fewer than 50% teachers log in during week 1
2. Risk: Data loss due to poor backup
   Mitigation: daily automated backup + restore testing
   Trigger: backup failure alert or missing backup file
3. Risk: Hosting downtime during peak hours
   Mitigation: reliable hosting + uptime monitoring
   Trigger: downtime more than 10 minutes

- **Current issues**

1. Issue: Attendance form is slow on mobile data
   Owner: frontend developer
   Due: Friday
   Action: optimize API calls, compress assets
2. Issue: Some colleges use different student ID formats
   Owner: data engineer
   Due: Wednesday
   Action: mapping table and validation rules